

ESPECIALIZACIÓN EN **INTELIGENCIA DE DATOS** para la Gestión Estratégica

FASES DE DISEÑO E INTEGRIDAD REFERENCIAL

Clase 2 · Taller T8002 — Representación de datos

Docente: Dr. Lautaro Di Bartolo

Asignatura: T8002 — Representación de datos

Cohorte o comisión: Cohorte 2026

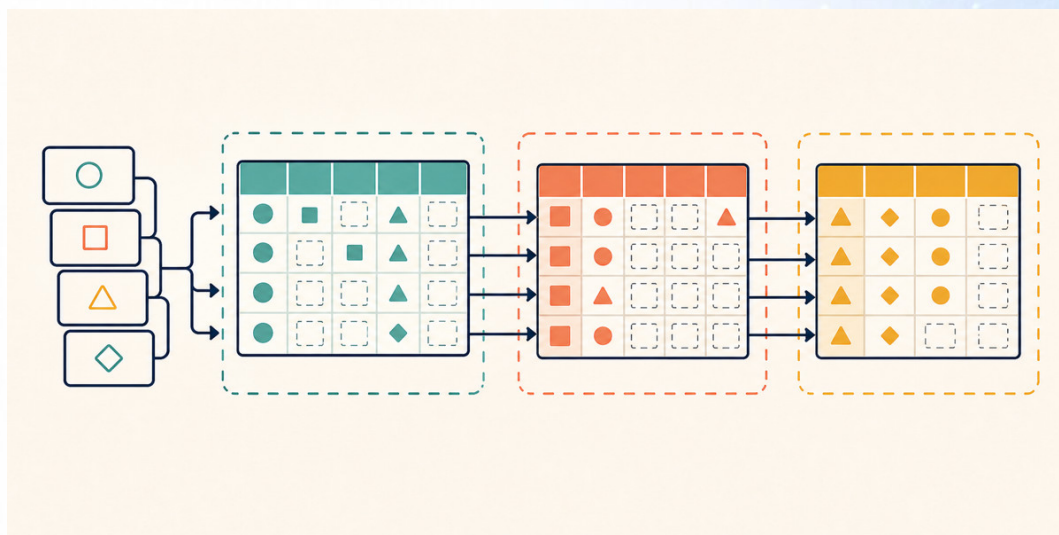


Figura 1. El diseño no empieza en la tabla: empieza en las preguntas que la tabla tiene que contestar.

1. INTRODUCCIÓN

1.1 Lo que la clase 1 dejó anotado

La clase 1 llegó a cuatro conclusiones, y esta clase las usa todas.

1. **La tabla plana tiene tres anomalías.** Modificar el teléfono de un encuestador obliga a tocar diecisiete filas. Pergamino no se puede cargar sin inventar una visita. Borrar las trece visitas a Concepción borra también que Concepción está en Tucumán.
2. **Una fila del relevamiento no es un hogar: es una visita.** Lo delató la clave, que es el par `id_hogar` más `fecha_visita`.
3. **Un archivo guarda valores y no guarda reglas.** El `personas = -3` entra sin una queja, y los dos hogares 41 del T8001 entraron igual.
4. **La salida está a la vista.** La sección 3.4 mostró dos tablas donde antes había una, y las tres anomalías desaparecieron.

Lo que la clase 1 **no** dio es el método. Mostró el destino de un pedazo del relevamiento y no explicó cómo se llega ahí desde cualquier tabla. Ese método es esta clase.

1.2 Lo que se supone de la clase 1

Esta clase supone seis palabras: **tabla**, **dominio**, **nulo**, **clave primaria**, **clave compuesta** y **clave foránea**. Si alguna no te suena, están en las secciones 5 y 6 del apunte anterior.

Dos avisos sobre repeticiones, para que no se lean como relleno.

La clave foránea aparece tres veces en el taller, y cada vez contesta otra pregunta. En la clase 1 fue *qué es*: una columna que guarda la clave primaria de otra tabla. Acá aparece dos veces más: en la sección 4.2, *de dónde sale* —qué vínculo del diseño la produce—, y en la sección 5.4, *qué obliga*, que es la integridad referencial.

Y las funciones del DBMS de la sección 7 de la clase 1 vuelven hoy. Esa sección se lee sola y se olvida rápido, así que conviene retener dos: **integridad**, que es el tema de la sección 5 de hoy, y **consulta**, que es lo que hace posible la sección 4.5.

1.3 El caso de esta clase

La misma tabla de la clase 1: relevamiento_plano.csv, 45 filas, 12 columnas, 37 hogares.

Hoy la vamos a mirar completa. Estas son sus doce columnas, agrupadas por lo que describen, y esa agrupación es el primer paso del trabajo del día:

Columna	¿De qué habla?
id_hogar	del hogar
localidad	del hogar, o de la localidad
provincia	de la localidad
fecha_visita	de la visita
encuestador_legajo	de la visita, y del encuestador
encuestador_nombre	del encuestador
encuestador_telefono	del encuestador
personas	de la visita
ingreso	de la visita
cobertura_salud	de la visita
satisfaccion	de la visita
servicios	del hogar

Leé la columna derecha de arriba abajo. Doce columnas hablan de **cuatro cosas distintas**: hogares, localidades, encuestadores y visitas. Y hay una quinta escondida, en servicios.

Esa lista es todo el diseño conceptual de hoy, en borrador. El resto de la clase es convertirla en un dibujo, y el dibujo en tablas.

2. LAS CUATRO FASES DEL DISEÑO

Diseñar una base de datos es una secuencia de cuatro pasos, y cada uno contesta una pregunta distinta. La confusión más común es saltar directo al cuarto.

Fase	Pregunta que contesta	Qué produce
Requisitos	¿Para qué va a servir?	una lista de preguntas y de reglas
Diseño conceptual	¿Qué cosas existen?	un diagrama de entidad y relación
Diseño lógico	¿Cómo quedan las tablas?	tablas, columnas, claves y dominios
Diseño físico	¿Cómo se guarda?	decisiones del administrador de la base

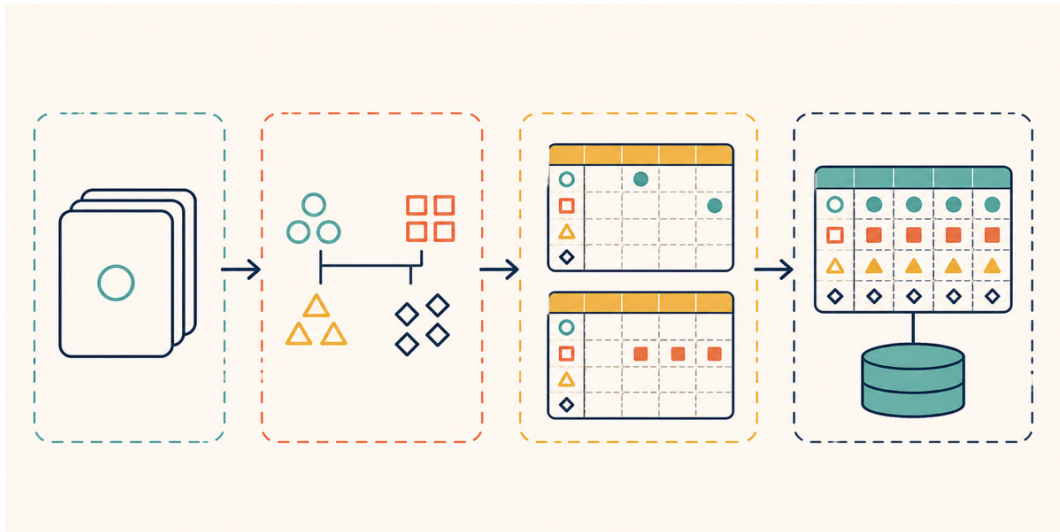


Figura 2. Cada fase contesta una pregunta distinta. Empezar por la última es tomar las tres primeras sin darse cuenta.

2.1 Requisitos: qué preguntas tiene que contestar la base

Antes de dibujar nada hay que escribir para qué sirve. Y "para guardar el relevamiento" no es un requisito: es una intención.

Un requisito es una pregunta concreta que la base tiene que poder contestar, o una regla que tiene que hacer cumplir. Los del relevamiento:

Preguntas que tiene que contestar

1. ¿Cuántos hogares se visitaron en cada localidad?
2. ¿Cómo cambió la cantidad de personas de un hogar entre su primera y su segunda visita?
3. ¿Cuántas visitas hizo cada encuestador, y en qué localidades?
4. ¿Qué proporción de hogares de cada provincia tiene cloacas?

Reglas que tiene que hacer cumplir

5. Un hogar pertenece a una sola localidad, y no puede cambiar de localidad.

6. Una visita corresponde a un hogar que existe y la hizo un encuestador que existe.
7. No puede haber dos visitas al mismo hogar el mismo día.
8. personas es un entero de 1 a 20. ingreso es mayor o igual a cero, o no se sabe.

Esas ocho líneas gobiernan todo lo que sigue. Fijate en la 4: pide cruzar provincia con servicios, que en la tabla plana son una columna repetida y una celda multivalor. Y fijate en la 7: es exactamente la clave compuesta que encontró la clase 1. **Un requisito bien escrito ya contiene una decisión de diseño.**

La fase que más se saltea, y la más cara de saltar. Sin requisitos escritos no hay forma de saber si un diseño es bueno: un diseño es bueno o malo respecto de las preguntas que tiene que contestar. Todos los diseños son correctos en el vacío.

2.2 Diseño conceptual: qué cosas existen

Acá se decide qué cosas del mundo van a existir en la base, qué propiedades tiene cada una y cómo se vinculan entre sí. El producto es un dibujo, y no menciona ninguna tabla, ningún tipo de dato y ningún nombre de columna.

Es la fase donde se discute con quien conoce el negocio, y por eso el resultado tiene que ser algo que esa persona pueda leer sin saber nada de bases de datos. Es la sección 3.

2.3 Diseño lógico: cómo quedan las tablas

Acá el dibujo se convierte en tablas, con sus columnas, sus claves primarias, sus claves foráneas y sus dominios. Se sigue sin hablar de ningún producto concreto: el resultado sirve igual para PostgreSQL que para SQL Server.

Es la fase donde vive la normalización de la clase 3, y son las secciones 4 y 5 de hoy. **Es donde este taller se detiene.**

2.4 Diseño físico, y por qué el orden no se puede saltar

El diseño físico decide cómo se guarda todo en el disco: qué índices se crean para acelerar las consultas frecuentes, cómo se reparte una tabla enorme, cuánta memoria se le asigna. Son decisiones de quien administra la base, dependen del producto que se use, y no cambian ni las tablas ni las reglas. El taller las nombra y sigue.

Y ahora, por qué el orden importa. Casi todo el mundo empieza por el final: abre una planilla y escribe encabezados. El problema es que ahí ya se tomaron, sin darse cuenta, todas las decisiones de las fases anteriores.

El relevamiento es la prueba. Alguien puso doce encabezados en una fila, y con eso decidió que un hogar, una localidad, un encuestador y una visita son la misma cosa. No lo decidió mal a propósito: no llegó a notar que estaba decidiendo. Las tres anomalías de la clase 1 son la factura de esa decisión invisible.

El orden sirve para eso: **para que cada decisión se tome cuando se la puede discutir.**

3. DISEÑO CONCEPTUAL

3.1 Entidad, atributo e instancia

Tres palabras, y las tres son fáciles:

- Una **entidad** es una clase de cosa sobre la que la base guarda datos. En el relevamiento: hogar, localidad, encuestador, visita y servicio.
- Un **atributo** es una propiedad de una entidad. La localidad tiene nombre y provincia. La visita tiene fecha, cantidad de personas e ingreso.
- Una **instancia** es un caso concreto de una entidad. Ramallo es una instancia de localidad.

Tercer homónimo, y el último. En la clase 1, sección 8.2, la instancia de una base de datos era su contenido en un momento dado, frente al esquema que es su estructura. Acá una instancia es un caso concreto de una entidad. Los dos sentidos son estándar, y se distinguen por lo que va antes: la instancia de una base es contenido, la instancia de una entidad es un caso.

La palabra "atributo" corrió un poco de lugar. En la clase 1 era una columna de una tabla. Acá es una propiedad de una entidad, y todavía no hay tablas. Son la misma idea vista desde dos fases: el atributo del diseño conceptual se va a convertir, en la sección 4, en una columna del diseño lógico. Casi siempre uno a uno, y no siempre.

3.2 Cómo se encuentra una entidad dentro de una tabla plana

La pregunta operativa es esta: **¿qué columnas se repiten siempre juntas?**

Mirá localidad y provincia en el relevamiento. Aparecen veinte veces Ramallo — Buenos Aires, doce veces Córdoba — Córdoba, trece veces Concepción — Tucumán. Nunca hay una fila con Ramallo y otra provincia. Se repiten siempre juntas y en las mismas combinaciones: son tres combinaciones distintas repartidas en 45 filas.

Ese patrón es el rastro de una entidad. Hay **tres localidades**, cada una con su provincia, y el archivo las está anotando cuarenta y cinco veces.

Lo mismo con las tres columnas del encuestador: cuatro combinaciones de legajo, nombre y teléfono, repartidas en 45 filas. Cuatro encuestadores.

La receta, entonces:

1. Buscá grupos de columnas que se repitan siempre juntas.
2. Contá cuántas combinaciones distintas hay. Si son muchas menos que las filas, encontraste una entidad.
3. La entidad se llama como la cosa que describen esas columnas, y sus atributos son esas columnas.

Esto tiene un nombre formal, y llega en la clase 3. "Columnas que se repiten siempre juntas" es la versión informal de una dependencia funcional. La clase 3 la define con precisión y la usa para verificar formalmente el diseño que hoy vamos a obtener a ojo. Hoy alcanza con el ojo.

3.3 Atributo o entidad: la decisión que más se equivoca

Acá está la decisión de la clase, y es una sola pregunta: ¿localidad es un **atributo del hogar** o una **entidad propia**?

Las dos respuestas son defendibles, y ahí está la trampa. Depende de tres cosas:

Preguntá	Si la respuesta es sí
¿Tiene propiedades propias?	entidad
¿Puede existir sin que haya ningún hogar en ella?	entidad
¿Necesito administrar la lista de valores válidos, agregando y quitando?	entidad

Para *localidad* las tres dan sí. Tiene una propiedad propia, la provincia. Pergamino puede existir sin tener todavía ningún hogar, y ese es el caso de la anomalía de inserción. Y la lista de localidades se administra: se agregan a medida que el operativo crece, que es lo que evita las siete formas de escribir tres localidades que encontró el T8001.

Contraejemplo, para ver la otra respuesta: *cobertura_salud* guarda si o no. No tiene propiedades propias, no existe con independencia del hogar y su lista de valores válidos son dos palabras que no se van a mover nunca. Es un **atributo**, y convertirlo en una tabla de dos filas solo agrega trabajo.

La regla corta. Si lo único que necesitás es limitar los valores posibles, alcanza un dominio. Si además esa cosa tiene datos propios o tiene que poder existir sola, es una entidad.

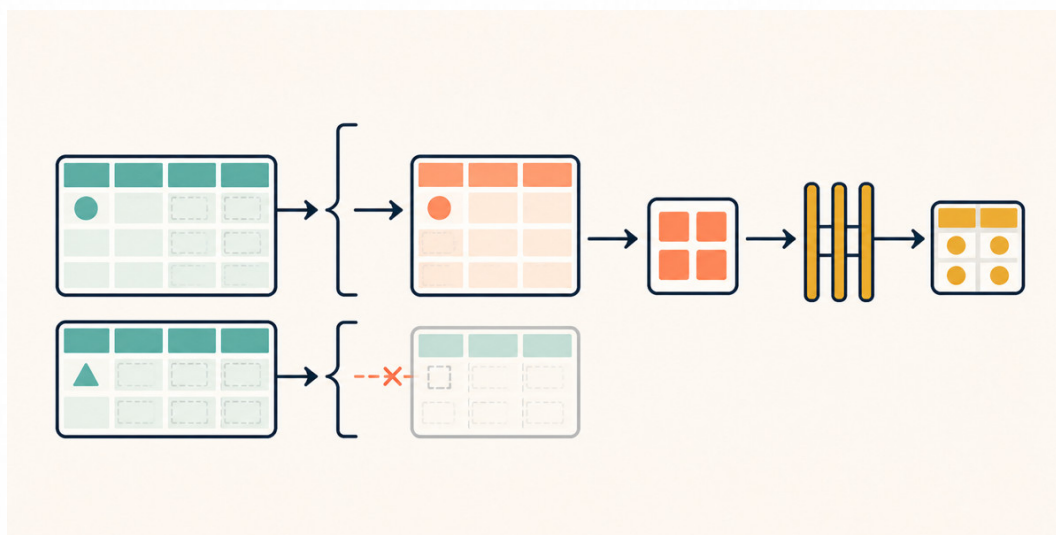


Figura 3. La misma columna puede ser atributo o entidad. Las tres preguntas deciden cuál.

3.4 Vínculo y cardinalidad

Segundo aviso sobre la palabra "relación". En la clase 1, sección 5.1, una relación era una tabla. En esta sección, una relación es un vínculo entre dos entidades, que en el dibujo es una línea. Los dos sentidos son estándar. Este apunte va a escribir vínculo para no tropezar.

Un vínculo es una conexión entre dos entidades: un hogar **pertenece a** una localidad, una visita **la hizo** un encuestador. La pregunta interesante no es si hay vínculo: es **cuántos de cada lado**. Eso se llama **cardinalidad**.

Se lee siempre en los dos sentidos, y conviene escribirlo así de literal:

*Un hogar pertenece a **una sola** localidad; una localidad tiene **muchos** hogares.*

Eso es un vínculo **uno a muchos**, y es el más frecuente de todos. Los tres casos:

Cardinalidad	Cómo se lee	Ejemplo del relevamiento
uno a uno	un A tiene un B, y un B tiene un A	ninguno
uno a muchos	un A tiene muchos B, un B tiene un solo A	localidad y hogar
muchos a muchos	un A tiene muchos B, y un B tiene muchos A	hogar y servicio

Los cuatro vínculos del relevamiento, escritos en los dos sentidos:

1. Un hogar pertenece a una sola localidad; una localidad tiene muchos hogares.
2. Una visita corresponde a un solo hogar; un hogar recibió muchas visitas.
3. Una visita la hizo un solo encuestador; un encuestador hizo muchas visitas.
4. Un hogar tiene muchos servicios; un servicio está en muchos hogares.

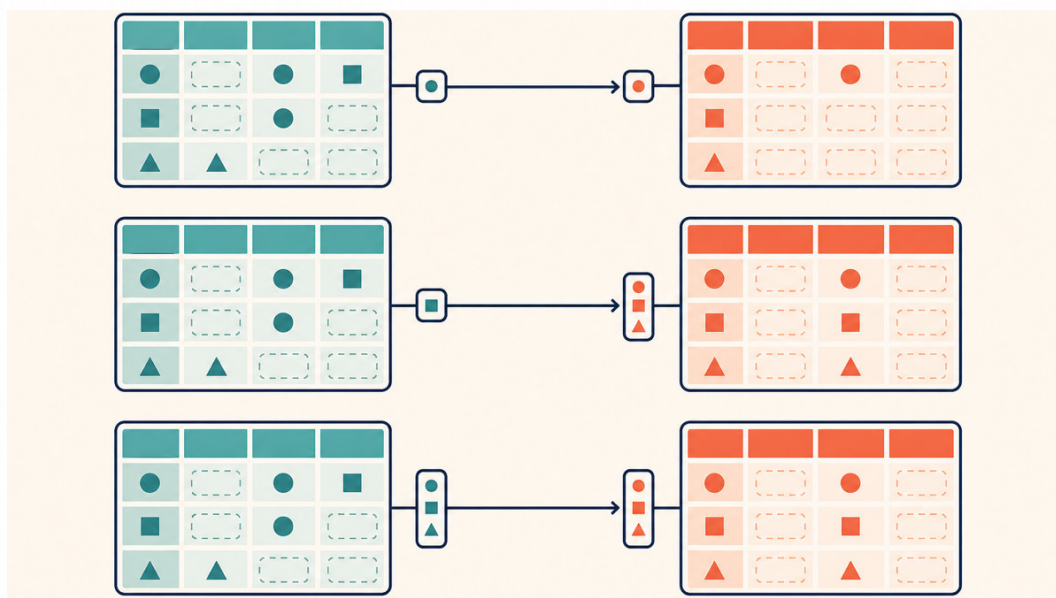


Figura 4. La cardinalidad se lee en los dos sentidos, y se decide por la regla y no por los datos.

Lo que la muestra sugiere no es la regla. La cardinalidad se decide preguntando por la regla, no mirando los datos. En estas 45 filas, veintinueve hogares tienen exactamente una visita: si mirás la tabla, el vínculo entre hogar y visita parece uno a uno. Ocho hogares alcanzan para desmentirlo. Pero incluso si ninguno tuviera dos, el vínculo seguiría siendo uno a muchos, porque el operativo permite revisitar. Con 45 filas casi cualquier cosa parece uno a uno. Preguntá si puede haber más, no si hay más.

3.5 Participación obligatoria y opcional

Una segunda pregunta sobre cada vínculo: ¿es obligatorio?

Un hogar **tiene que** pertenecer a una localidad: no existe un hogar sin localidad. Eso es participación **obligatoria**. Una localidad **puede** no tener ningún hogar todavía, y ese es el caso de Pergamino: participación **opcional**.

La participación obligatoria u opcional es lo que resuelve la anomalía de inserción. Si el vínculo de localidad a hogar fuera obligatorio en los dos sentidos, Pergamino seguiría sin poder existir.

Y acá aparece la respuesta a un problema que el T8001 dejó abierto, que es el mejor argumento de todo el taller para diseñar bien.

El T8001 encontró diez filas donde `cobertura_salud` y `satisfaccion` estaban las dos vacías, en las mismas diez filas. No pudo decidir si era un módulo opcional que el encuestador podía omitir o un salto condicional del formulario, y las dejó como estaban. Diez nulos, sin explicación.

Con diseño, eso tiene salida. Si esas dos preguntas son un **módulo opcional**, entonces no son atributos de la visita: son una entidad aparte, con participación **opcional** desde la visita. Una visita puede tener un módulo o no tenerlo.

El efecto es este: **la fila existe solamente cuando la pregunta correspondía**. Los diez nulos no se rellenan y no se borran. Dejan de existir, porque no hay fila donde ponerlos. Y la distinción entre "no aplica" y "no contestó", que el T8001 perdió al declarar todo como nulo, vuelve a ser visible: si no hay fila, no aplicaba; si hay fila con el valor vacío, correspondía y no contestó.

Y ahora la decisión de este taller: **el esquema de la sección 4.4 no aplica esa salida**. Deja las dos columnas en `visita`, con nulos permitidos. La razón es que nadie confirmó la regla: no sabemos si es un módulo opcional o un salto condicional del formulario, y separar una entidad por una regla sin confirmar es exactamente el error que la clase 3 advierte en su sección 5.4. La

salida existe y queda anotada; falta la pregunta a quien organizó el relevamiento. Es un buen ejemplo de que un diseño se traba en una regla del negocio, y no en la técnica.

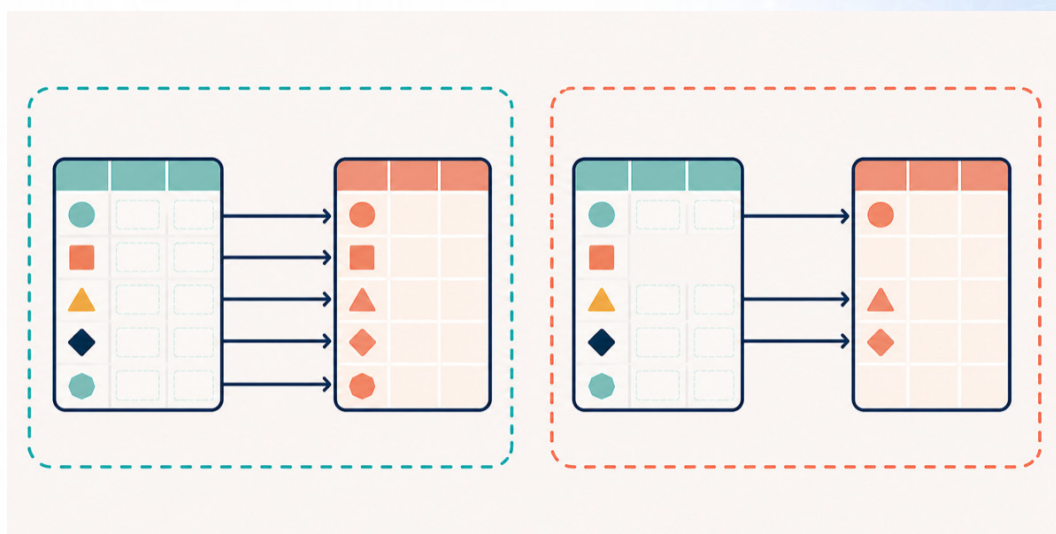


Figura 5. Cuando el vínculo es opcional, la ausencia de fila es un dato y no un agujero.

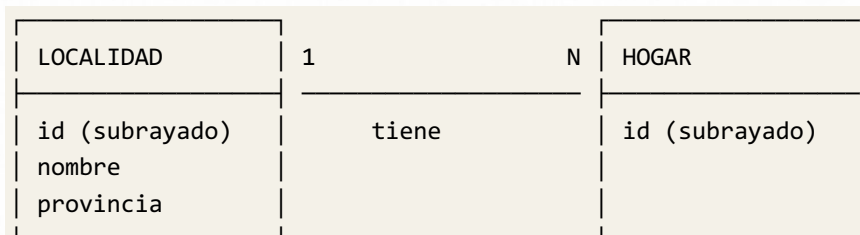
3.6 Cómo se dibuja un diagrama de entidad y relación

El producto del diseño conceptual es un dibujo llamado **diagrama de entidad y relación**, o DER. La convención que usa este taller es la mínima que sirve, y son cuatro elementos:

1. **Una caja por entidad**, con el nombre en singular arriba y los atributos debajo.
2. **Una línea por vínculo**, entre las dos cajas que conecta.
3. **La cardinalidad escrita en cada extremo de la línea**: 1 o N.
4. **El atributo que identifica va subrayado**.

Y una convención más para la participación: la línea llena es obligatoria, la línea punteada es opcional. Si te resulta más cómodo, escribí "obligatorio" al lado. Es lo mismo.

Ejemplo mínimo, con el primer vínculo del relevamiento:

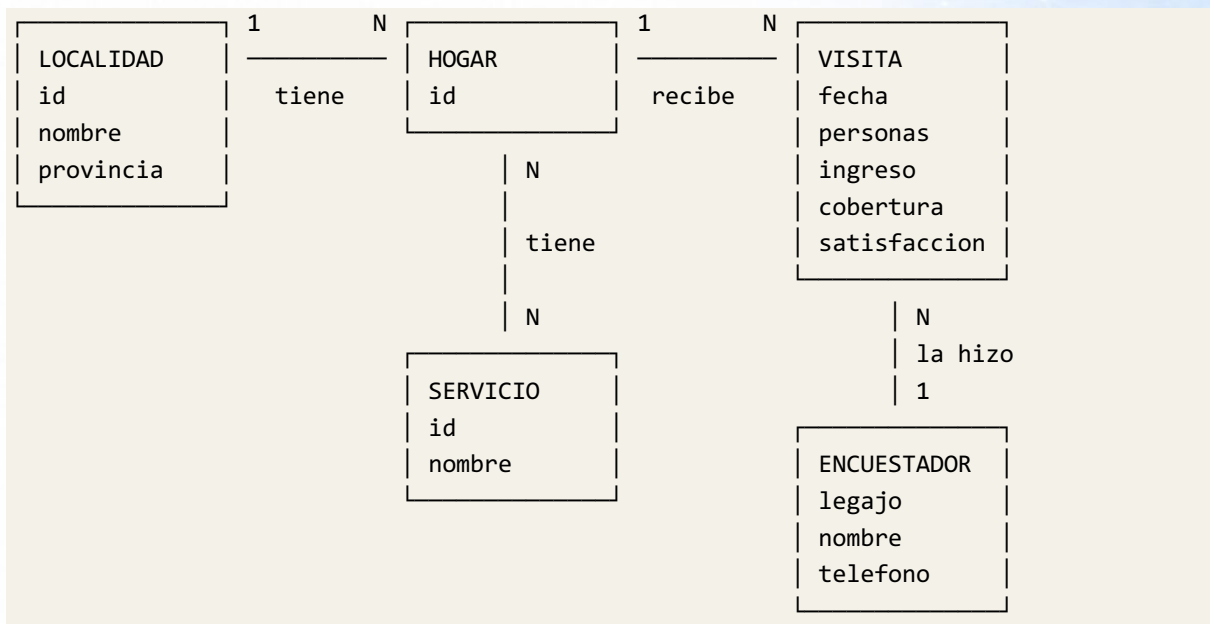


Se lee: una localidad tiene N hogares, y un hogar pertenece a 1 localidad.

Hay muchas notaciones y ninguna es la correcta. Vas a encontrar rombos para los vínculos (Chen), patas de gallo en los extremos (crow's foot), y la notación de UML. Todas dicen lo mismo. En este taller la notación no se evalúa, ni en clase ni en el trabajo final: se evalúa que las entidades y las cardinalidades sean las correctas. Un DER a mano, en papel, fotografiado, vale exactamente igual.

3.7 El diagrama del relevamiento

Con las cinco entidades de la sección 1.3 y los cuatro vínculos de la sección 3.4:



Como en un bloque de texto no se puede subrayar, el atributo que identifica va **primero** en cada caja: id en LOCALIDAD, HOGAR y SERVICIO, legajo en ENCUESTADOR. VISITA no tiene identificador propio: se identifica por el hogar más la fecha, y eso aparece en la regla 4 de la sección 4.1.

Cinco entidades, cuatro vínculos. Tres son uno a muchos y uno, el de hogar con servicio, es muchos a muchos. Ese último es el que tiene una consecuencia especial, y es la sección 4.3.

Fijate lo que **no** hay en el dibujo: no hay tipos de dato, no hay decisiones de almacenamiento, y no hay ninguna columna que apunte a otra tabla. HOGAR tiene solo su id, y en ningún lado dice localidad_id. La línea y su 1/N ya guardan esa información, y la columna que la va a materializar aparece en la fase siguiente, con la regla 2 de la sección 4.2.

4. DE LAS ENTIDADES A LAS TABLAS

4.1 Las cuatro reglas para pasar a tablas

El pasaje del diagrama a las tablas es mecánico. Cuatro reglas alcanzan para casi todos los casos:

Regla	Qué se hace
1	Cada entidad se convierte en una tabla. Sus atributos son las columnas, y el atributo subrayado es la clave primaria.
2	Cada vínculo uno a muchos pone una clave foránea en el lado "muchos".
3	Cada vínculo muchos a muchos crea una tabla nueva, con las dos claves foráneas.
4	Si una entidad no se identifica sola, su clave primaria incluye la clave foránea que la identifica. visita se identifica por el hogar más la fecha.

Que sea mecánico es la recompensa de haber hecho bien el diseño conceptual. **Si el pasaje resulta difícil, el problema está en el diagrama y no en el pasaje.**

4.2 Uno a muchos: la clave viaja al lado muchos

La regla 2 es la que más se equivoca, y siempre en la misma dirección: se pone la clave foránea del lado equivocado.

Vínculo: una localidad tiene muchos hogares. ¿Dónde va la referencia?

- **Mal:** una columna hogar_id en localidad. Ramallo tiene quince hogares, así que habría que guardar quince valores en una celda. Es la celda multivalor que el T8001 pasó media clase 3 desarmando.
- **Bien:** una columna localidad_id en hogar. Cada hogar tiene **una** localidad, así que en la celda va un solo valor.

De ahí sale la regla en una frase: **la clave foránea va del lado que tiene uno.** Y ese lado es el que en el diagrama está marcado con N, que es el que confunde. La forma segura de decidirlo es preguntar del lado de la fila: "un hogar, ¿cuántas localidades tiene?". Una. Entonces la columna va en hogar.

Aplicado a los tres vínculos uno a muchos del relevamiento:

- hogar recibe localidad_id.
- visita recibe hogar_id.
- visita recibe encuestador_legajo.

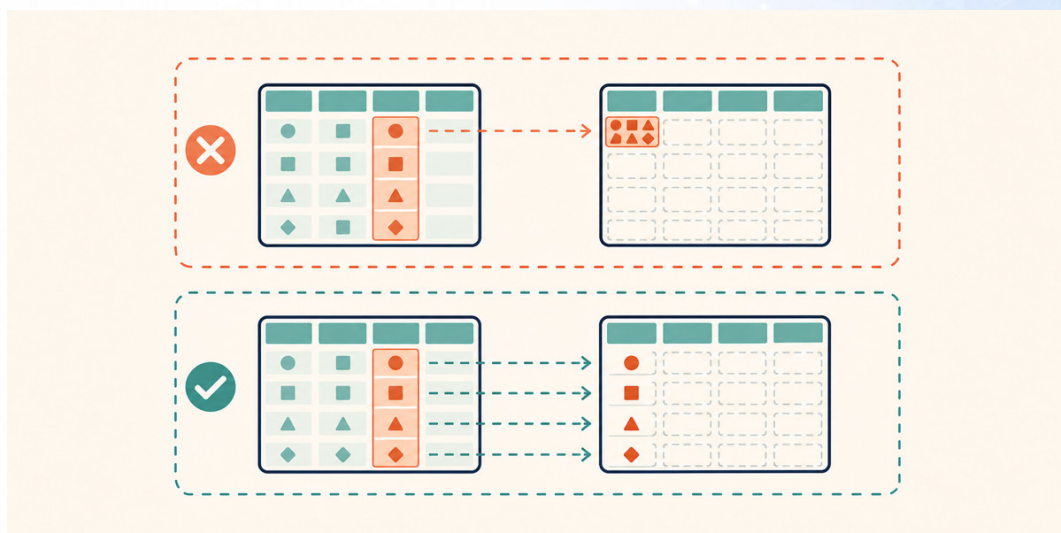


Figura 6. La referencia va donde cabe en una sola celda. Del otro lado no cabe.

4.3 Muchos a muchos: aparece una tabla nueva

Un hogar tiene muchos servicios, y un servicio está en muchos hogares. Ninguna de las dos tablas puede guardar la referencia en una celda: la regla 2 falla en los dos sentidos.

La regla 3 dice qué hacer: crear una **tabla nueva**, que existe solo para conectar. Cada fila es un par.

Tabla hogar_servicio

hogar_id	servicio_id
1	1
1	2
1	3
2	1
2	2

El hogar 1 tiene agua, luz y gas: tres filas. En total son **88 filas**, una por cada par de hogar y servicio. Su clave primaria es el **par** de las dos columnas, que es una clave compuesta como la de la clase 1. Y las dos columnas son, cada una, clave foránea.

Esta es la tabla que resuelve la columna servicios del relevamiento, que guardaba agua;luz;gas;cloacas en una celda. Y notá una cosa: la tabla nueva puede tener **atributos propios**. Si hiciera falta guardar desde cuándo el hogar tiene cloacas, esa fecha no es del hogar ni del servicio: es del par. Va acá.

Cuándo la tabla nueva merece un nombre propio. Si el par tiene atributos propios, casi siempre es que había una entidad escondida. hogar_servicio con una fecha de conexión es en realidad una conexión de servicio. Ponerle el nombre bueno ayuda a que el diseño se lea.

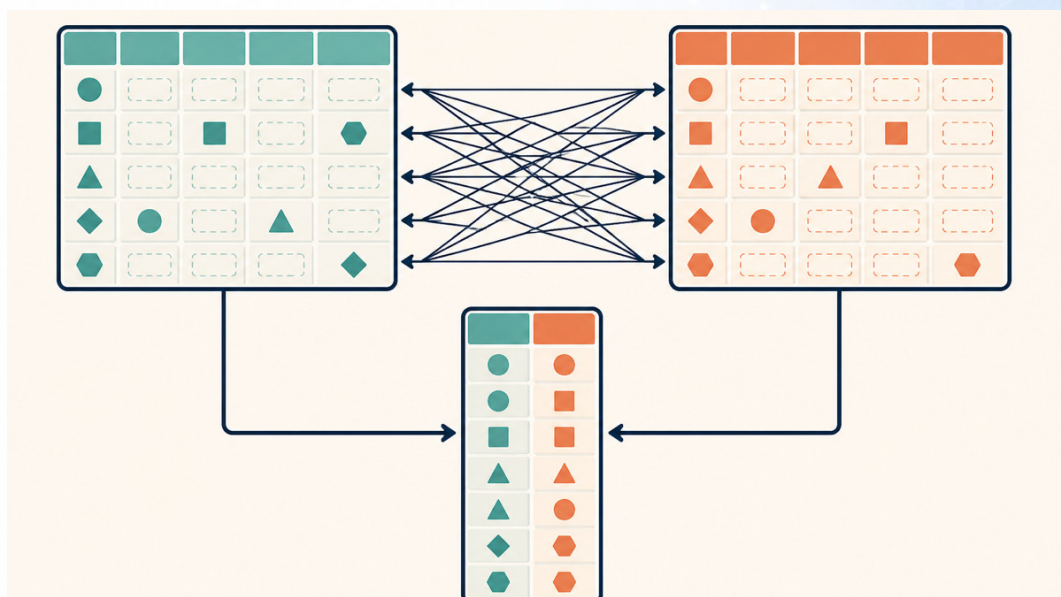


Figura 7. Cuando ninguno de los dos lados puede guardar la referencia, aparece una tabla que solo existe para conectar.

4.4 El esquema completo del relevamiento

Las seis tablas. La clave primaria va en **negrita** y la clave foránea lleva una flecha con su destino:

localidad

Columna	Dominio
id	entero positivo
nombre	texto, obligatorio
provincia	texto, obligatorio
(nombre, provincia)	único: el par no se puede repetir

encuestador

Columna	Dominio
legajo	texto de 3 caracteres
nombre	texto
telefono	texto

hogar

Columna	Dominio
id	entero positivo
localidad_id → localidad.id	entero, obligatorio

visita

Columna	Dominio
hogar_id → hogar.id	entero, obligatorio
fecha	fecha desde 2025-03-01
encuestador_legajo → encuestador.legajo	texto, obligatorio
personas	entero de 1 a 20
ingreso	número ≥ 0, o nulo
cobertura_salud	si o no, o nulo
satisfaccion	mala, regular o buena, o nulo

servicio

Columna	Dominio
id	entero positivo
nombre	uno de: agua, luz, gas, cloacas

hogar_servicio

Columna	Dominio
hogar_id → hogar.id	entero, obligatorio
servicio_id → servicio.id	entero, obligatorio

Seis tablas donde había una. Y acá conviene desarmar una creencia habitual, porque los números no dicen lo que se espera:

	Filas	Celdas
Tabla plana	45	540
Las seis tablas juntas	$3 + 4 + 37 + 45 + 4 + 88 = 181$	594

El diseño normalizado ocupa **más** celdas, no menos. Normalizar no es comprimir, y a esta escala casi nunca ahorra espacio: las claves foráneas y las filas de hogar_servicio cuestan lugar.

Lo que se gana es otra cosa, y es lo único que importa: **cada hecho está escrito una sola vez**. La columna provincia pasó de 45 celdas a 3, y la del teléfono del encuestador, de 45 a 4. Visto por hecho: que Ramallo está en Buenos Aires estaba escrito veinte veces, y ahora está escrito una vez. Las tres anomalías de la clase 1 vivían ahí, en la diferencia entre veinte y uno.

Fijate en visita: su clave primaria es el par hogar_id más fecha. Es exactamente la clave compuesta que la clase 1 encontró a los tanteos, y acá sale de la regla 4 sin buscarla. Cuando el diseño está bien, las claves no hay que adivinarlas.

4.5 Volver a juntar las tablas: el cruce

Separar es la mitad del trabajo. La otra mitad es poder volver a juntar, porque si no se puede, separar rompió el dato en vez de organizarlo.

La operación que junta dos tablas se llama **cruce**, o *join*. Funciona así: se eligen dos tablas y la columna por la que se corresponden, y el resultado es una tabla nueva donde cada fila de una se completa con los datos de la otra.

Cruzar hogar con localidad por `localidad_id`:

hogar.id	hogar.localidad_id		localidad.nombre	localidad.provincia
1	1	→	Ramallo	Buenos Aires
2	2	→	Córdoba	Córdoba
3	3	→	Concepción	Tucumán

Y ahí está la propiedad importante: **cruzando las seis tablas se recupera la tabla plana original, fila por fila y valor por valor**. No se perdió nada. El dato es el mismo; lo que cambió es dónde vive cada hecho.

Eso tiene dos consecuencias que valen para el resto del taller:

- Es la **prueba** de que una descomposición está bien hecha. Si al volver a juntar aparecen filas que no estaban, o faltan filas que estaban, la descomposición está mal. La clase 3 usa esta prueba, y el trabajo final también.
- Es la respuesta a "¿y entonces cómo miro la tabla completa?". Se cruza. En una base de datos eso es una consulta de una línea, y se puede guardar con nombre para no repetirla. Eso es una **vista**, y llega en la clase 3.

En el notebook, el cruce se hace con `merge` de `pandas`. Es la única función de `pandas` que este taller necesita explicar, y hace justo esto.

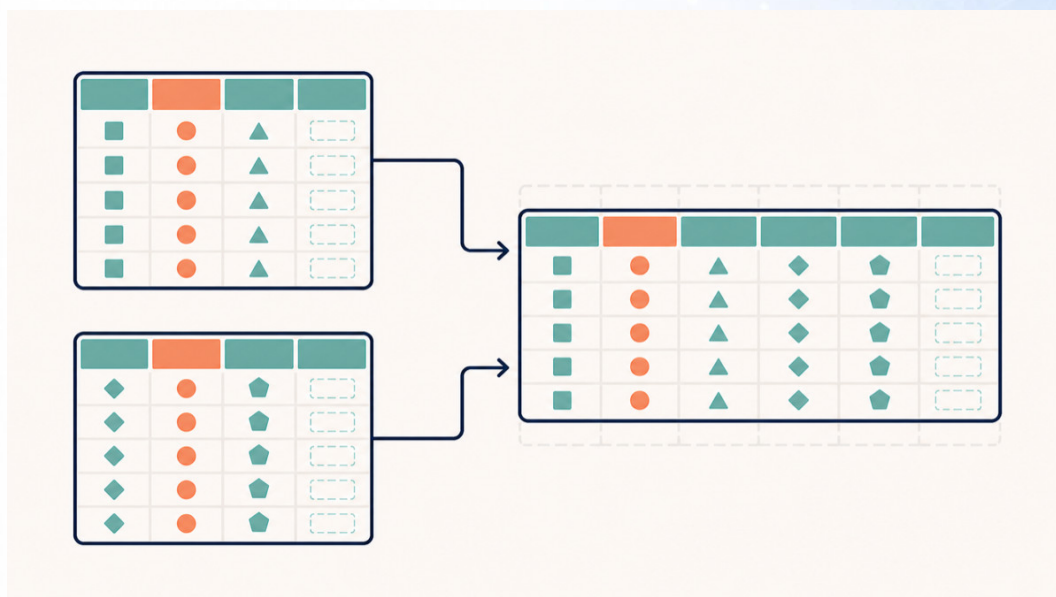


Figura 8. Si el cruce no devuelve la tabla original, la separación estaba mal. Es la prueba, no un detalle.

4.6 Qué preguntas contesta ahora el diseño

Todo lo anterior fue trabajo. Conviene cobrarlo. Volvamos a los ocho requisitos de la sección 2.1:

Requisito	En la tabla plana	En las seis tablas
1. Hogares por localidad	se puede, contando con cuidado los repetidos	una cuenta directa sobre hogar
2. Cambio de personas entre visitas	se puede	directo: dos filas de visita del mismo hogar
3. Visitas y localidades por encuestador	se puede	directo, con un cruce
4. Cloacas por provincia	no se puede sin desarmar la celda multivalor	directo, con dos cruces
5. Un hogar, una localidad	nada lo impide	lo garantiza la columna sola más la clave foránea
6. Visita de un hogar que existe	nada lo impide	lo garantiza la integridad referencial
7. No dos visitas el mismo día	nada lo impide	lo garantiza la clave primaria de visita
8. Dominios de personas e ingreso	nada lo impide	lo garantiza el dominio declarado

Leé la última columna. Los requisitos 1 a 4 pasaron de "se puede con cuidado" a "es directo". Y los requisitos 5 a 8 pasaron de "nada lo impide" a "**lo garantiza el diseño**", que es una diferencia de otra naturaleza: dejaron de depender de que alguien se acuerde.

Con una salvedad en el requisito 5, que pide dos cosas. Que un hogar tenga **una sola** localidad lo garantiza que `localidad_id` sea una columna sola, y que esa localidad **exista** lo garantiza la clave foránea. Que el hogar no pueda **cambiar** de localidad no lo garantiza ninguna de las dos: un UPDATE de esa columna se acepta. Eso necesita una regla aparte, y es el tipo de detalle que aparece al implementar.

Y hay una ganancia más, que no está en la lista y es la que más importa a largo plazo. **Un dato que vive en un solo lugar se puede auditar.** El teléfono del encuestador E04 está en una celda: se puede decir quién lo cargó, cuándo y qué decía antes. Cuando el mismo teléfono está en diecisiete filas, no hay a qué preguntarle. El T8001 llamó a eso trazabilidad, y dijo que era la dimensión de calidad más subestimada. Esta es la razón por la que se pierde.

Lo que se paga: para ver la tabla completa hay que cruzar, y son seis tablas para entender en vez de una. Es un costo real, y la clase 3 discute cuándo vale la pena no pagarlo.

5. QUÉ SE DECLARA AL CREAR UNA BASE, Y QUÉ QUEDA PROHIBIDO

5.1 El esquema es una declaración de qué se admite

Hasta acá el diseño es un documento. **Crear la base de datos** es declararlo de una forma que el DBMS pueda hacer cumplir.

Y ese es el cambio de naturaleza de toda la clase. En una planilla, las reglas de la sección 2.1 son un acuerdo: están escritas en algún lado y se cumplen si todos se acuerdan. En una base de datos son **parte del esquema**, y el DBMS las revisa en cada carga, sin excepciones y sin que nadie se lo pida.

Así se declara la tabla `localidad`. Son seis renglones en SQL:

```
CREATE TABLE localidad (  
  id          INTEGER PRIMARY KEY,  
  nombre      TEXT    NOT NULL,  
  provincia   TEXT    NOT NULL,  
  UNIQUE (nombre, provincia)  
);
```

No hay que aprender esa sintaxis y no se va a pedir nunca en este taller. Lo único que importa es reconocer las piezas: **PRIMARY KEY** es la clave primaria, **NOT NULL** es obligatorio, **UNIQUE** es que no se repite, y **TEXT** e **INTEGER** son el tipo. Y **UNIQUE** sobre dos columnas dice que el **par** no se repite: es lo que deja entrar dos Concepciones de provincias distintas, que es justo el caso con el que la clase 1, sección 6.2, justificó la clave sustituta. Cuatro de las reglas que venimos discutiendo, en seis renglones. Escribirlas es el contenido de Modelado de datos.

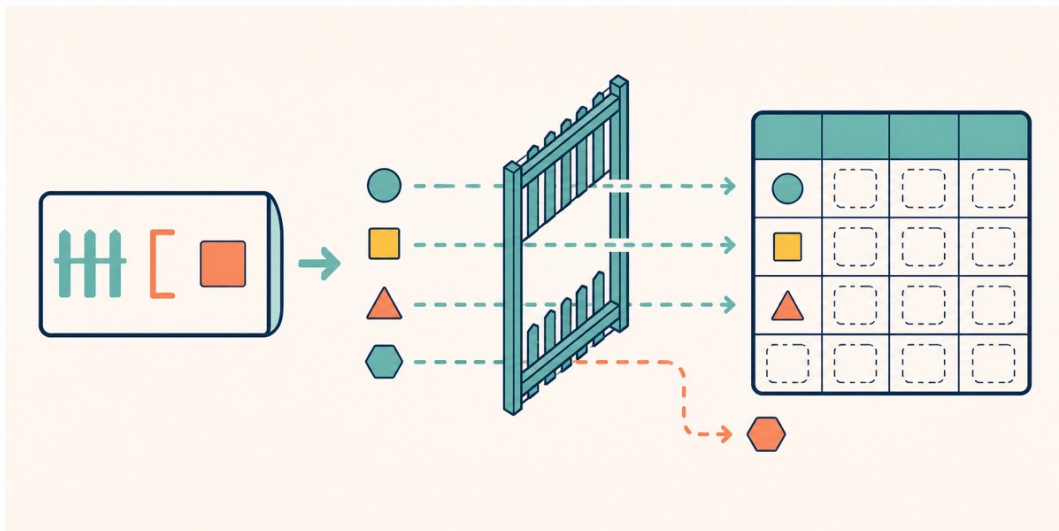


Figura 9. El documento de reglas se vuelve un cerco activo: lo que no lo cumple, no entra.

5.2 Tipos y dominio

Lo primero que se declara de una columna es su **tipo**: entero, número con decimales, texto, fecha. El tipo es la primera restricción de dominio, y es gruesa: impide poner *hola* donde va un número, y no impide poner *-3* donde van personas.

Para el resto del dominio hay tres declaraciones más:

Declaración	Qué impide	Ejemplo del relevamiento
rango	valores fuera de un intervalo	personas entre 1 y 20
lista de valores	cualquier valor fuera de la lista	satisfaccion: mala, regular o buena
valor por omisión	que la columna quede sin dato cuando la carga no lo manda	fecha de visita, la fecha de hoy

Las cuatro declaraciones juntas —tipo, rango, lista de valores y valor por omisión— son la **integridad de dominio**: la garantía de que ningún valor imposible entra a una columna.

Comparalo con el T8001. Ese taller detectó el 999999 de la columna ingreso **después**, mirando un promedio inflado un 35%, y tuvo que deducir que era un centinela porque no había diccionario que lo dijera. Con el dominio declarado, el 999999 no habría entrado: la carga lo habría rechazado en el momento, con la persona que tenía el formulario adelante.

La diferencia no es que la base revise: es cuándo. Un archivo acepta el dato malo y el problema aparece meses después, disfrazado de resultado raro. Una base lo rechaza cuando todavía hay alguien que sabe cuál era el valor bueno.

5.3 Obligatorio y único: la integridad de entidad

Dos declaraciones más, y de las dos sale la primera de las tres integridades.

Obligatorio quiere decir que la columna no admite nulos. **Único** quiere decir que el valor no se puede repetir en la tabla.

La clave primaria es las dos cosas a la vez, siempre, y no se puede elegir lo contrario. Esa regla tiene nombre: **integridad de entidad**. Toda fila tiene que ser identificable, y una fila con la clave vacía o repetida no lo es.

Y acá cierra, del todo, el caso del hogar 41 del T8001, que la clase 1 contó en su sección 6.5.

Con *id* declarado como clave primaria de hogar, la segunda fila **no entra**. La carga falla, en el momento, con el formulario en la mano. Y el duplicado exacto que ese taller también tuvo que buscar y borrar tampoco entra, por la misma razón.

Es el mismo dato, el mismo defecto y el mismo operativo. Lo que cambia es el momento en que el error se descubre, y de eso depende si se puede arreglar.

5.4 La clave foránea y la integridad referencial

Tercera aparición de la clave foránea, y la que dice **qué obliga**.

La declaración de una clave foránea trae una regla puesta: **todo valor de esa columna tiene que existir en la tabla a la que apunta**. Eso es la **integridad referencial**.

Lo que impide, en concreto, es la **fila huérfana**: una fila que se refiere a algo que no existe. Una visita con `hogar_id = 99`, sin que haya ningún hogar 99. En la tabla plana eso ni se nota, porque no hay ninguna otra tabla que contradiga nada. Con la clave foránea declarada, esa carga falla.

La huérfana aparece por tres caminos, y los tres son cotidianos:

- Un error de tipeo en la carga: `hogar_id = 99` en lugar de 9.
- Un borrado que se llevó la localidad y dejó los hogares que la referenciaban.
- Una carga masiva que se hizo en el orden equivocado, cargando visitas antes que hogares.

Las tres integridades, juntas, para tenerlas en una tabla:

Integridad	Qué garantiza	Con qué se declara
de dominio	ningún valor imposible	tipo, rango, lista de valores
de entidad	ninguna fila sin identidad	clave primaria: obligatoria y única
referencial	ninguna referencia al vacío	clave foránea

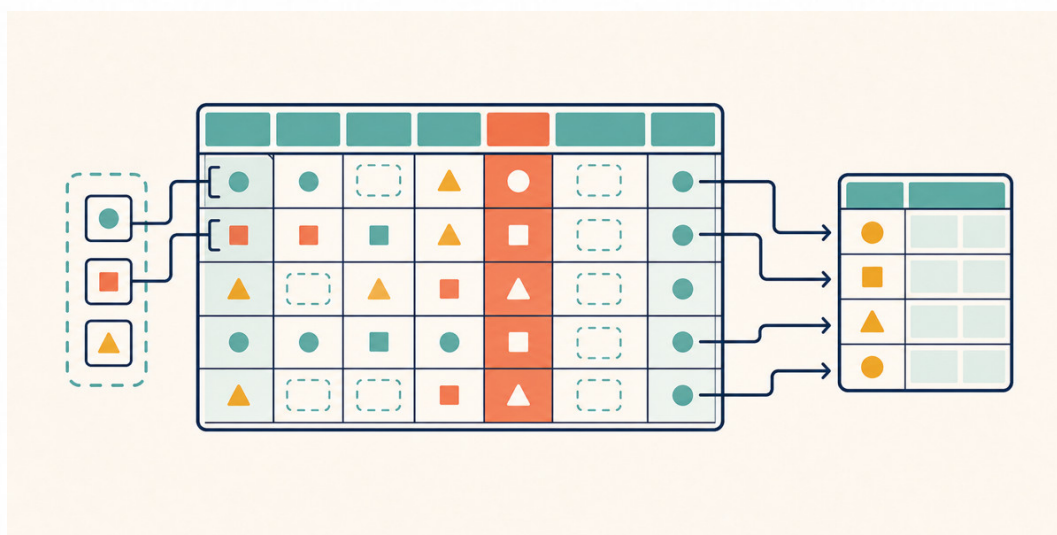


Figura 10. Tres integridades, tres preguntas distintas. Una tabla puede cumplir dos y fallar la tercera.

5.5 Qué pasa al borrar una localidad que tiene hogares

Si alguien intenta borrar la localidad Ramallo, y hay quince hogares que la referencian, ¿qué tiene que pasar?

No hay una respuesta correcta: hay tres, y se elige una al declarar la clave foránea.

Opción	Qué hace	Cuándo se usa
Bloquear	falla mientras queden hogares que la referencien	es el valor por omisión, y casi siempre el correcto
Propagar	borra la localidad y sus quince hogares	solo cuando la fila que referencia no tiene sentido sola
Anular	borra la localidad y deja los hogares con la referencia vacía	cuando el vínculo es opcional

Para localidad y hogar, la respuesta es **bloquear**. Un hogar sin localidad no tiene sentido, y borrar quince hogares porque alguien quiso corregir el nombre de una localidad es una catástrofe silenciosa.

Para hogar y hogar_servicio, en cambio, **propagar** es razonable: si el hogar deja de existir, sus conexiones de servicio no significan nada.

La opción peligrosa es propagar, y es la que más se elige por comodidad. Un borrado que propaga mal declarado puede vaciar media base con una sola operación que parecía chica. La duda se resuelve siempre del mismo lado: bloquear.

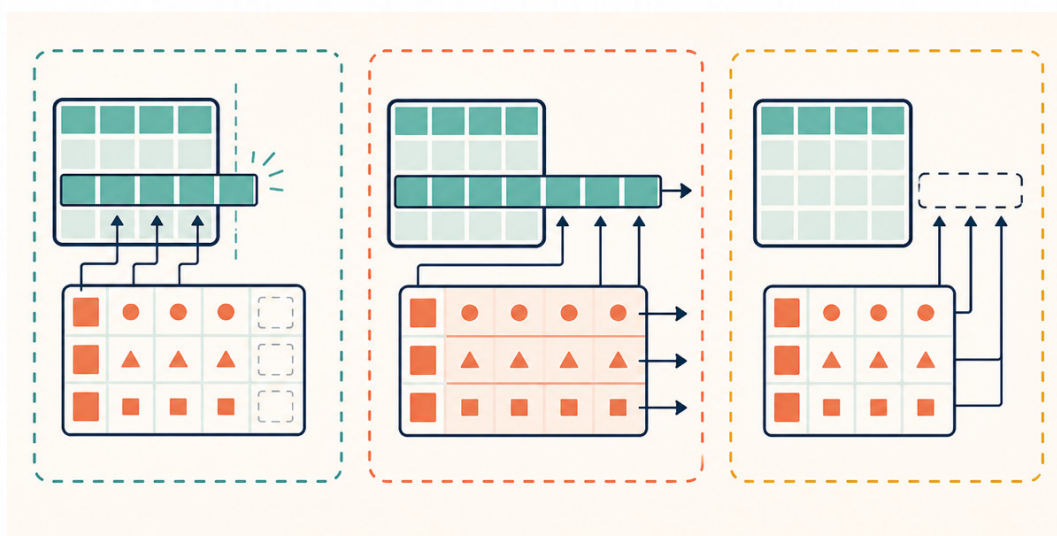


Figura 11. Tres desenlaces para el mismo borrado. El que no se elige es el que queda por omisión.

5.6 La fila que la base rechaza es una buena noticia

Al cargar el relevamiento en las seis tablas, algunas filas van a fallar. Y eso es lo mejor que puede pasar.

Un rechazo no es un problema del diseño: es el diseño **funcionando**. Cada fila rechazada es un defecto que hasta hoy estaba adentro de los datos, contando en los promedios, sin que nada avisara.

Es el mismo giro que el T8001 hizo con la conversión de fechas que fallaba: el bloque rojo no era el problema, era la evidencia. Acá vale igual, y con una ventaja: el rechazo dice exactamente qué regla se violó y en qué fila.

La actitud correcta frente a un rechazo, en tres pasos:

1. **Leé qué regla se violó.** El mensaje lo dice: clave repetida, dominio violado, referencia inexistente.
2. **Decidí quién está mal: el dato o la regla.** Las dos cosas pasan. Si el dato dice personas = 25 y la regla dice hasta 20, capaz el dato es un error de carga, y capaz existe un hogar de 25 personas y el dominio estaba mal puesto.
3. **Anotá la decisión.** Es lo mismo que el T8001 pedía para cada decisión de limpieza, y por la misma razón.

El paso 2 es el que se saltea. **Una regla declarada también puede estar mal**, y aflojar una regla para que entre un dato es una decisión legítima, siempre que sea una decisión y no un reflejo.

5.7 Lista de verificación

Antes de dar por cerrado un diseño, siete preguntas:

1. **¿Escribí los requisitos antes del diagrama?** Si no, no hay con qué juzgar el diseño.
2. **¿Cada entidad pasa las tres preguntas de la sección 3.3?** Propiedades propias, existencia independiente, lista de valores que hay que administrar.
3. **¿Cada cardinalidad está escrita en los dos sentidos?** Y decidida por la regla, no por los datos que tenés.
4. **¿Cada vínculo dice si es obligatorio u opcional?** Es lo que resuelve la anomalía de inserción y los nulos con significado.
5. **¿Cada clave foránea está del lado que tiene uno?** Si una celda tendría que guardar varios valores, está del lado equivocado.
6. **¿Cruzando las tablas se recupera la tabla original?** Es la única prueba de que no se perdió nada.
7. **¿Cada dominio está declarado, y cada borrado tiene su opción elegida?** Si no elegiste, elegiste el valor por omisión sin saberlo.

6. LOS EJEMPLOS EN GOOGLE COLAB

El notebook de esta clase sigue el orden del apunte, en seis bloques. En clase los vamos a ejecutar juntos, alternando con la presentación:

1. **Las cuatro cosas de las que habla la tabla.** Contar combinaciones distintas, y ver aparecer las tres localidades y los cuatro encuestadores de la sección 3.2.
2. **Atributo o entidad.** Las tres preguntas de la sección 3.3, aplicadas a localidad y a cobertura_salud.
3. **La cardinalidad no se lee en los datos.** Los veintinueve hogares con una sola visita, y los ocho que desmienten la conclusión fácil de la sección 3.4.
4. **Las seis tablas.** Construir las a partir de la tabla plana, y mirar cuántas filas quedó teniendo cada una.
5. **El cruce.** Reconstruir la tabla plana con merge y comprobar, celda por celda, que es la misma que entró.
6. **Las filas que la base rechaza.** Un personas = -3, una visita con hogar_id = 99 y un segundo hogar 41, con la regla que cada uno viola.

No hay nada que completar: [02_diseño_e_integridad.ipynb](#).